

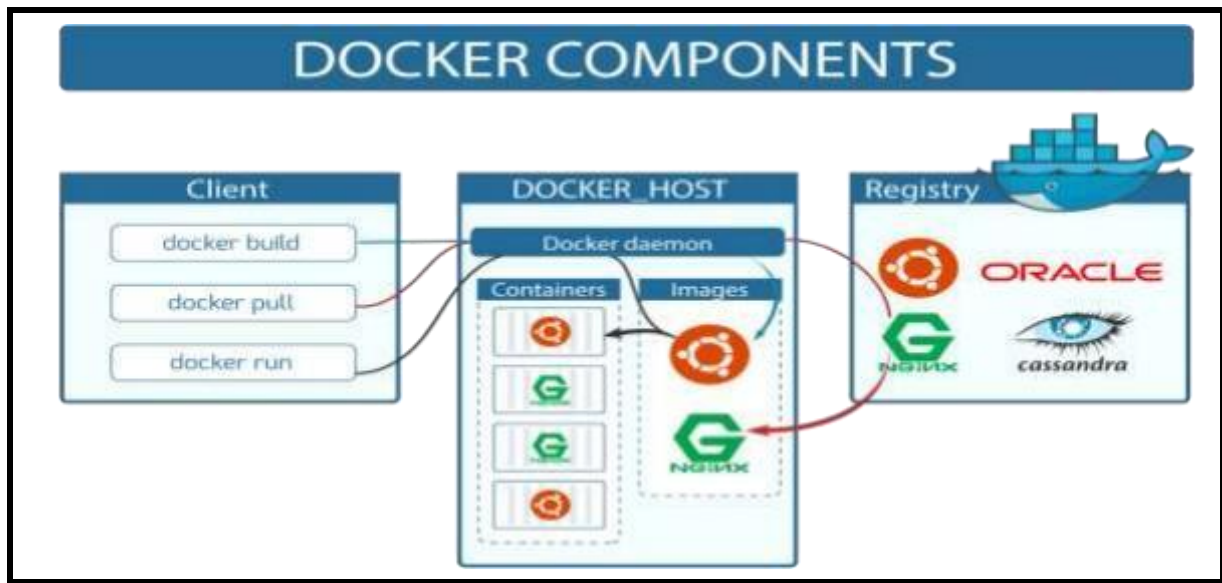
SECTION-05

9.a) Discuss the Components of Docker container.

Docker is an open-source software platform. It is designed to make it easier to create, deploy, and run applications by using containers. Containers allow a developer to package up an application with all of the parts which are required, such as libraries and other dependencies and ship it all out as one package.

Docker Components

These are the Docker Components:



DOCKER CLIENT

The Docker client enables users to interact with Docker.

Docker runs in a client-server architecture that means docker client can connect to the docker host locally or remotely.

Docker client and host (daemon) can run on the same host or can run on different hosts and communicate through sockets or a RESTful API.

The Docker client is the primary way that many Docker users interact with Docker. When we use commands such as docker run, the client sends these commands to docker daemon, which carries them out. The docker command uses the Docker API. The Docker client can communicate with more than one daemon.

We can communicate with the docker client using the Docker CLI. We have some commands through which we can communicate the Docker client. Then the docker client passes those commands to the Docker daemon.

docker build ... docker run ... docker push ..etc.

DOCKER HOST

An image can be used to build a container. Container images can be shared across teams within an enterprise using a private container registry, or shared with the world using a public registry Docker Hub.

Docker Containers:

A container is a runnable instance of an image. We can create, start, stop, move, or delete a

container using the Docker API or CLI. We can connect a container to one or more networks, attach storage to it, or even create a new image based on its current state.

Docker Networking

Through the docker networking, we can communicate from one container to other containers.

By default, we get three different networks on the installation of Docker — none, bridge, and host. The none and host networks are part of the network stack in Docker. The bridge network automatically creates a gateway and IP subnet and all containers that belong to this network can talk to each other via IP addressing.

Docker Storage

A container is volatile it means whenever we remove or kill the container then all of its data will be lost from it. If we want to persist the container data use the docker storage concept.

We can store data within the writable layer of a container but it requires a storage driver. In terms of persistent storage, Docker offers the following options:

Data Volumes

Data-Volume Container

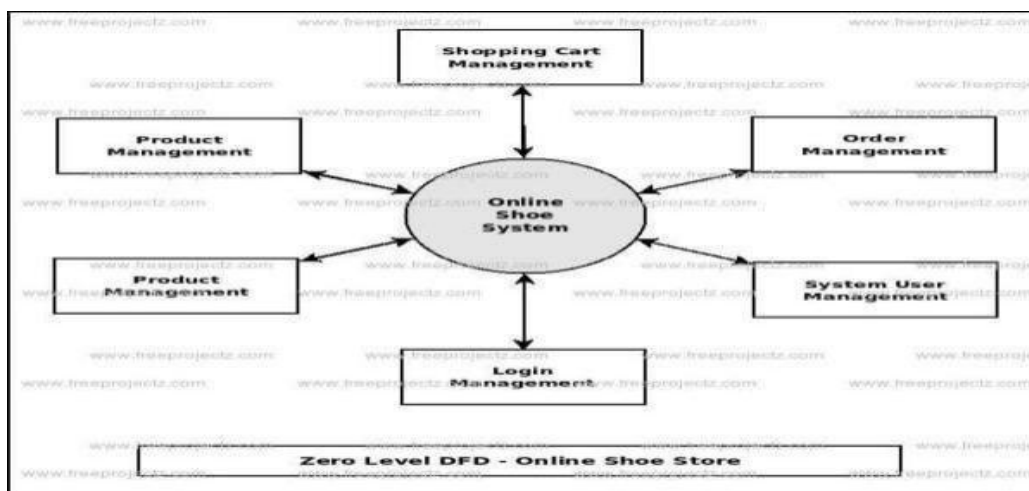
Bind Mounts

DOCKER REGISTRIES

Docker-registries are services that provide locations from where we can store and download images. A Docker registry contains repositories that host one or more Docker Images. Public Registries include Docker Hub and Docker Cloud and private Registries can also be used. We can also create our own private registry. Push or pull image from docker registry using the following commands `docker push` `docker pull` `docker run`.

Draw the CI/CD build process flow diagram for an online footwear store application and explain each component.

CI/CD Build Process Flow - Online Footwear Store Application. The CI/CD pipeline for an online footwear store is designed to streamline development, ensure code quality, and automate deployment. It consists of several interconnected stages that handle everything from code commits to production release.



Source Code Repository

Developers write code (React, Java, TypeScript).

Code is stored in GitHub or GitLab.

Version Control System

Tracks changes in code.

Triggers pipeline when new code is pushed.

CI Server (e.g., Jenkins, GitHub Actions)

Starts the build process automatically.

Pulls latest code from the repository.

Build Stage

Installs dependencies.

Compiles code and prepares the app.

Automated Testing

Runs unit tests (e.g., for login, cart, payment).

Checks if the app works correctly.

Artifact Storage

Saves the built app (e.g., Docker image, JAR file).

Ready for deployment.

Deployment (CD)

Sends the app to staging or production servers.

Can be manual or automatic.

Monitoring & Feedback

Tracks errors, performance, and user activity.

Helps improve future releases.

You want to have two versions of your application in production, but be able to switch all traffic between them. Explain the deployment strategy suitable for the given situation.

Blue/green deployment is a deployment technique to release new code into the production environment. Blue/green deployments make use of two identical production environments one of these is active and the other environment is set to idle. New updates are pushed to the active environment where it is monitored for bugs while the idle environment serves as a backup where traffic can be routed in case an error occurs.

Blue/green deployment provide the following benefits to businesses

Simple, fast, easy to understand and implement.

Rollback is straightforward as teams need to simply flip traffic back to the old environment in case any issue arises.

Blue-green deployments are not as risky and vulnerable to losses as compared to other deployment strategies.



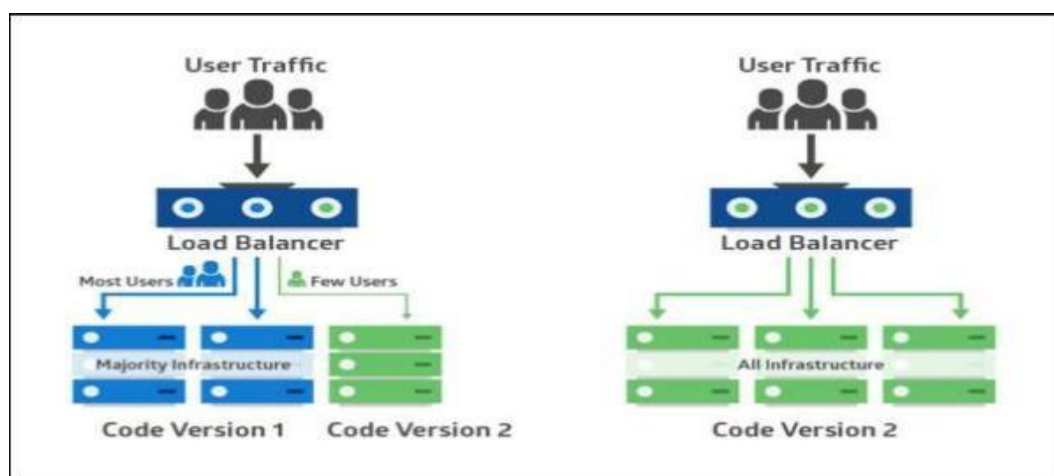
Canary deployment is a technique to reduce the risk of updating software or introducing new changes in production environment by slowly rolling out the change to a small subset of users before making the software functional for everyone. Canary deployments provide the following benefits to businesses.

- Allows enterprises to test in production with real users and use cases.

- Enables comparison of different service versions side by side.

- Cheaper than blue-green deployments because it does not require two production environments.

- DevOps team can rapidly and safely trigger a rollback to a previous version of an Application



10.a) Create a form to add a new product detail to the product catalogue using React. React component code for Productform.js

```
import React, { useState } from 'react';
```

```
function ProductForm() {
```

```
  const [product, setProduct] =
```

```
    useState({ name: "",
```

```
    category: "",
```

```
    price: "
```

```
  });
```

```
  const handleChange = (e) => { const
```

```
    { name, value } = e.target;
```

```

        setProduct(prevState => ({
            ...prevState,
            [name]: value
        }));
    };
    const handleSubmit = (e) =>
    { e.preventDefault();
      console.log('New Product Details:', product);
      // Here you can add code to send data to a server or update state
    };
    return (
      <form onSubmit={handleSubmit}>
        <label>
          Product Name:
        />
        <input type="text" name="name" value={product.name} onChange={handleChange}
        />
        </label>
        <br />
        <label>
          Product Category:
        />
        <input type="text" name="category" value={product.category}
        onChange={handleChange} />
        </label>
        <br />
        <label>
          Price:
        />
        <input type="number" name="price" value={product.price}
        onChange={handleChange} />
        </label>
        <br />
        <button type="submit">Add Product</button>
      </form>
    );
  }
  export default ProductForm;

```

b) Ted is a DevOps engineer. He wants to automate the deployment process. He wants to do deployment frequently on multiple servers and change the CPU speed and RAM based on the requirement.

To automate deployments across multiple servers and dynamically adjust CPU and RAM based on requirements, Ted should implement Infrastructure as Code (IaC) using tools like Terraform or AWS CloudFormation. This allows him to define server configurations—including CPU speed and memory—in code, making it easy to provision or modify infrastructure as needed. For frequent deployments, integrating a CI/CD pipeline with Jenkins or GitHub Actions ensures that every code change is automatically built, tested, and deployed across environments.

To manage deployments on multiple servers, configuration management tools like Ansible or container orchestration platforms like Kubernetes can be used. These tools help push updates consistently and handle scaling. For dynamic resource allocation, cloud platforms such as AWS or Azure offer autoscaling features. Ted can set policies that automatically increase or decrease CPU and RAM based on traffic or performance metrics. Monitoring tools like Prometheus or CloudWatch can track usage and trigger scaling events, ensuring optimal performance without manual intervention.

This setup enables fast, reliable deployments and efficient resource management, aligning perfectly with modern DevOps practices.

Process and Human Intervention

Automated deployment relies on software tools, scripts, and systems to carry out the deployment process. Developers and operations teams set up deployment pipelines that automatically execute predefined steps, reducing the need for direct human intervention.

Speed and Efficiency

Automated Deployment: Automated deployment is generally much faster and more efficient. Once the deployment pipeline is set up, it can be executed repeatedly without human intervention. This consistency ensures that the deployment process is reliable and predictable.

Consistency and Reproducibility

Automated deployment ensures consistency and reproducibility. The same deployment process is applied across all environments, reducing the risk of configuration drift and errors.

Risk and Error Minimization

Automated systems can help minimize human errors as the deployment process is predefined and thoroughly tested. Automated rollback mechanisms can also be implemented to revert to a stable state in case of deployment issues.

Scalability and Complexity

Automation can handle more complex deployments and scale easily as it is designed to handle a wide range of scenarios. It is well-suited for modern microservices-based architectures and cloud-native applications.

Continuous Deployment and Continuous Integration (CI/CD)

Automated deployment is a key enabler for CI/CD workflows, where changes are automatically tested, integrated, and deployed to production, often multiple times a day.

Deployment Automation Strategy for Ted

Ted is a DevOps engineer aiming to streamline deployments across multiple servers while adjusting CPU and RAM based on application needs. To achieve this, he can use Infrastructure as Code (IaC) tools like Terraform or AWS CloudFormation, which allow him to define server specs—including CPU speed and memory—in reusable configuration files. This makes it easy to spin up or modify servers as needed.

Template-Based Server Provisioning

Ted can create reusable templates for different server types—like high-performance nodes for payment processing and lightweight ones for static content. This allows him to quickly spin up servers with predefined CPU and RAM specs depending on the workload.

Environment Consistency Across Servers

Using containerization tools like Docker, Ted can ensure that the application runs the same way on every server, regardless of the underlying hardware. This reduces configuration drift and simplifies multi-server deployments.

Scheduled and Event-Driven Deployments

Ted can configure deployments to run on a schedule (e.g., nightly builds) or trigger them based on specific events—like a successful merge or a spike in traffic. This gives him flexibility to deploy proactively or reactively, depending on business needs.